SR-MPLS Training Lab

M5 Quality of Service

Estimated time: 60 min

Objective

Give SR-MPLS a working class-of-service chain so a premium flow keeps its priority end to end across the core. You will classify customer traffic by DSCP at the ingress PE, map it to a forwarding class, rewrite that intent into MPLS EXP bits on every transit link, and prove the marking survives with queue counters and a packet capture. By the end you can trace one packet from its DSCP value at the PE-CE edge to its EXP value on a core link.

Before you start: M4 complete: Flex-Algo and the L3VPN overlay are up, both Linux clients are reachable inside CUST_A (Client1 10.50.1.10 behind PE1, Client2 10.50.2.10 behind PE2), and the client image has iperf3 and tcpdump installed.

Concept primer

Class of service in this lab is a marking chain, not a bandwidth guarantee. Customer traffic arrives at the ingress PE carrying a DSCP value in its IP header. A behavior-aggregate (BA) classifier on the PE-CE subinterface reads that DSCP and assigns the packet to a forwarding class: DSCP 46 (EF, code-points 101110) becomes premium-priority, and everything else (code-points 000000) becomes best-effort. The forwarding class is the router's internal name for the service level; it is what every downstream decision keys off. Once the packet is pushed into an MPLS label stack for transport across the core, its original IP DSCP is no longer what core routers look at, so the intent has to be copied into the label header. That is the job of the EXP (also called TC) bits, three bits in every MPLS label. A rewrite rule on each transit interface writes premium-priority as EXP 5 (code point 101) and best-effort as EXP 0 (code point 000), and an EXP classifier on the far end reads those bits back into a forwarding class, so the service level rides hop by hop with the labels.

The stage5 config builds this in named pieces. 'forwarding-classes' bind queue 0 to best-effort and queue 5 to premium-priority. Two schedulers describe how each class is served: BE-SCHEDULER takes 70 percent transmit rate and buffer, PREMIUM-SCHEDULER is priority strict-high with 30 percent buffer. The

scheduler-map SR-MPLS-SMAP ties each forwarding class to its scheduler, and that map is applied to every transit interface. The EXP classifier SR-MPLS-EXP-CLASS and rewrite rule SR-MPLS-EXP-REWRITE are applied to unit 0 of each transit interface so the core reads incoming EXP and re-marks outgoing EXP consistently. The extra piece that lives only on the PE is the DSCP classifier DSCP-INGRESS, applied to the PE-CE subinterfaces ge-0/0/3 unit 100 and unit 200, which is where customer DSCP first becomes a forwarding class. Core routers such as P1 carry only the EXP classifier, rewrite, and scheduler-map because they never see customer IP headers, only labels.

Read the proof honestly. vJunos-router runs a virtualized lite-mode PFE ('set chassis fpc 0 lite-mode' in every base config), so classification, forwarding-class assignment, EXP rewrite, and queue counters are configuration-faithful and observable, but hardware-accurate scheduling, shaping, and tail-drop under real congestion are not reproduced. In plain terms: this module proves the marking and classification chain is correct (the right packets land in the right forwarding class and leave with the right EXP bits), and it proves that with queue counters and a capture. It does not prove that premium traffic would actually be protected by starving best-effort during congestion, because the lite-mode PFE does not enforce the schedulers the way silicon would. Keep that line clear when you brief: correct marking, yes; real drop behavior, no.

Build steps

1. **Define the two forwarding classes and their queues (all nodes).**

```
set class-of-service forwarding-classes queue 0 best-effort
set class-of-service forwarding-classes queue 5 premium-priority
```

Note: queue 0 is best-effort, queue 5 is premium-priority. These names are what every classifier, scheduler, and rewrite rule below refers to. The same two lines appear on PE and core nodes.

2. **Define the schedulers and bind them in a scheduler-map (all nodes).**

```
set class-of-service schedulers BE-SCHEDULER transmit-rate percent 70
set class-of-service schedulers BE-SCHEDULER buffer-size percent 70
set class-of-service schedulers PREMIUM-SCHEDULER priority strict-high
set class-of-service schedulers PREMIUM-SCHEDULER buffer-size percent 30
set class-of-service scheduler-maps SR-MPLS-SMAP forwarding-class best-effort scheduler BE-SCHEDULER
set class-of-service scheduler-maps SR-MPLS-SMAP forwarding-class premium-priority scheduler PREMIUM-SCHEDULER
```

Note: PREMIUM-SCHEDULER is priority strict-high so premium drains before best-effort. On the lite-mode PFE this describes intent faithfully but is not enforced as real congestion behavior.

3. **Define the EXP classifier and EXP rewrite rule (all nodes).**

```
set class-of-service classifiers exp SR-MPLS-EXP-CLASS forwarding-class premium-priority loss-priority low code-points 101
set class-of-service classifiers exp SR-MPLS-EXP-CLASS forwarding-class best-effort loss-priority low code-points 000
set class-of-service rewrite-rules exp SR-MPLS-EXP-REWRITE forwarding-class premium-priority loss-priority low code-point 101
set class-of-service rewrite-rules exp SR-MPLS-EXP-REWRITE forwarding-class best-effort loss-priority low code-point 000
```

Note: The classifier reads incoming EXP into a forwarding class; the rewrite writes the forwarding class back out as EXP. Premium is EXP 5 (code point 101), best-effort is EXP 0 (code point 000).

4. **Apply the scheduler-map, EXP classifier, and EXP rewrite to each transit interface. PE1 has two transit links (ge-0/0/1, ge-0/0/2); a core node like P1 has three (add ge-0/0/3).**

```
set class-of-service interfaces ge-0/0/1 scheduler-map SR-MPLS-SMAP
set class-of-service interfaces ge-0/0/1 unit 0 classifiers exp SR-MPLS-EXP-CLASS
set class-of-service interfaces ge-0/0/1 unit 0 rewrite-rules exp SR-MPLS-EXP-REWRITE
set class-of-service interfaces ge-0/0/2 scheduler-map SR-MPLS-SMAP
set class-of-service interfaces ge-0/0/2 unit 0 classifiers exp SR-MPLS-EXP-CLASS
set class-of-service interfaces ge-0/0/2 unit 0 rewrite-rules exp SR-MPLS-EXP-REWRITE
```

Note: The scheduler-map binds to the physical interface; the classifier and rewrite bind to unit 0. On P1 repeat the same three lines for ge-0/0/3 (the plane cross-link). Each node's exact interface set is in configs/stage5-qos/<node>.set.

5. **PE ONLY: add the DSCP ingress classifier and apply it to the PE-CE subinterfaces. This is where customer DSCP first becomes a forwarding class.**

```
set class-of-service classifiers dscp DSCP-INGRESS forwarding-class premium-priority loss-priority low code-points 101110
set class-of-service classifiers dscp DSCP-INGRESS forwarding-class best-effort loss-priority low code-points 000000
set class-of-service interfaces ge-0/0/3 unit 100 classifiers dscp DSCP-INGRESS
set class-of-service interfaces ge-0/0/3 unit 200 classifiers dscp DSCP-INGRESS
commit and-quit
```

Note: DSCP 101110 is 46 decimal (EF) and maps to premium-priority; 000000 is best-effort. Unit 100 is CUST_A, unit 200 is CUST_B. Core routers never get this classifier because they only see labels. Push the whole stage per node with 'python push_lab_configs.py stage5-qos'.

6. **On Client1 (Linux, 10.50.1.10 in CUST_A), generate an EF flow to Client2, then a best-effort flow, so the PE has something to classify.**

```
client1$ iperf3 -c 10.50.2.10 --tos 184 -t 20    ## --tos 184 = 0xB8 = DSCP 46 (EF)
client1$ iperf3 -c 10.50.2.10 -t 20              ## no --tos = best-effort (DSCP 0)
```

Note: --tos takes the full 8-bit ToS byte: 184 decimal is DSCP 46 in the high six bits, which the DSCP-INGRESS classifier maps to premium-priority. The plain run defaults to DSCP 0, which maps to best-effort. Run each for long enough to watch counters move.

7. **In GNS3, capture a core link and read the MPLS EXP bits in Wireshark while the EF flow runs.**

```
## Right-click the P1 to PE1 link (plane A) in GNS3 and choose 'Start capture'.
## Re-run the EF iperf3 from Client1 so premium packets cross the link during capture.
## Open the capture in Wireshark and expand MultiProtocol Label Switching Header.
## Read the 'MPLS Experimental Bits' (TC) field on the EF flow's frames.
```

Note: The capture is the ground-truth data-plane proof: on the EF flow the MPLS EXP field reads 5, on the best-effort flow it reads 0. This is what queue counters alone cannot show you.

Verification

show class-of-service interface ge-0/0/1

```
Physical interface: ge-0/0/1, Index: 151
  Scheduler map: SR-MPLS-SMAP, Index: 57302
  Logical interface: ge-0/0/1.0, Index: 343
  Rewrite-Output   SR-MPLS-EXP-REWRITE   exp (mpls-any)
  Classifier       SR-MPLS-EXP-CLASS     exp
```

What to look for: The physical interface shows Scheduler map SR-MPLS-SMAP, and unit 0 lists both the EXP Rewrite-Output and the EXP Classifier. A missing Rewrite-Output line here is exactly this module's break/fix: MPLS frames would leave with EXP 000.

show class-of-service rewrite-rule name SR-MPLS-EXP-REWRITE

```
Rewrite rule: SR-MPLS-EXP-REWRITE, Code point type: exp
  Forwarding class          Loss priority    Code point
```

```
best-effort            low           000
premium-priority       low           101
```

What to look for: premium-priority at low loss priority maps to code point 101 (EXP 5) and best-effort to 000. If premium mapped to 000, every transit router would queue the premium flow as best-effort no matter what the PE classified.

show interfaces queue ge-0/0/1

```
Queue: 0, Forwarding classes: best-effort
  Transmitted: ... packets  (best-effort flow counts here)
Queue: 5, Forwarding classes: premium-priority
  Transmitted: ... packets  (EF flow counts here)
```

What to look for: While the EF iperf3 runs, the Transmitted counter on queue 5 (premium-priority) climbs; the best-effort run moves queue 0. This confirms classification put each flow in the right queue. Remember the lite-mode PFE reports placement faithfully but does not enforce real scheduling or drops.

GNS3 capture of the P1 to PE1 link opened in Wireshark

```
MultiProtocol Label Switching Header
    Label: 16002 (PE2 node-SID)
    Experimental Bits: 5      <- EF flow
    Experimental Bits: 0      <- best-effort flow
```

What to look for: The EF flow's frames carry MPLS EXP = 5 on the core link, the best-effort flow carries EXP = 0. This is the end-to-end proof that the DSCP-to-forwarding-class-to-EXP chain survived the push into the label stack. It is the one check the router CLI cannot give you on its own.

[screenshot 006: M5 verification: MPLS EXP marking on a core-link capture]

Break / Fix exercise

Scenario: Customers report that the premium (EF) flow inside CUST_A behaves no differently from best-effort once it is more than one hop into the core. Every IS-IS adjacency is Up, the L3VPN is healthy, there is no session alarm, and the ingress PE still classifies EF correctly. But a packet capture on a core link shows the premium flow leaving one transit interface with MPLS EXP set to 000 instead of 101, and from that hop onward downstream routers queue everything as best-effort.

Your task: Find the single transit interface that is no longer marking premium traffic into EXP 5 and restore correct EXP marking across the core, using only class-of-service configuration. Confirm the fix in a capture, not just in the CLI.

Hint: The forwarding-class assignment is fine; what is missing is the step that copies that assignment into the MPLS label header on egress. Compare 'show class-of-service interface' on each transit interface against a known-good one and look for the object that is present on the healthy interface but absent on the faulty one. Queue counters will not flag this, because queue placement is set at ingress, not by the egress marking; only the capture and the interface's applied objects reveal it.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Trace one EF packet from Client1 to Client2: name the DSCP value it carries at the PE-CE edge, the forwarding class it is assigned, and the MPLS EXP value it carries on a core link, and say which config object performs each of those steps.
- Explain why a missing EXP rewrite on just one transit link collapses service differentiation for every router downstream of that link, even though the ingress PE classified the traffic correctly and no session alarms fire.
- State precisely what the vJunos lite-mode PFE does and does not prove in this module: which parts of the class-of-service chain are configuration-faithful and observable, and which behavior (real scheduling, shaping, and tail-drop under congestion) it cannot reproduce.